Queens College, CUNY, Department of Computer Science
CSCI 212 Lab - Object Oriented Programming in Java
In Lab Coding Assignment 2

Racing Simulation Project

Objective

In this project, you will build a simplified Java racing simulation. The project is divided into two parts.

Part I – In-Lab Race Simulation: 60%

You will complete the core racing simulation during the 50-minute lab period.

Part II – Safety Car Extension: 40%

After the lab, you will extend your program by adding a `SafetyCar` class and Safety Car behavior.

Your Part II program should **extend the code you submitted during Part I**, rather than replacing it with a completely different program.

Race Rules

The race consists of:

- **11 teams**
- **2 drivers per team**
- **22 drivers total**
- **55 laps**

Each team has its own:

- range of possible lap times
- range of possible pit-stop times
- range of possible retirement rates

Each driver on the same team uses that team's ranges.

Qualifying

Before the race begins, each driver completes **one qualifying lap**. The driver's qualifying time is generated randomly using the driver's team lap-time range. Drivers are then ranked from the smallest qualifying time to the largest.

At the start of the race:

P1 starts at 0.0 seconds
P2 starts at 0.5 seconds
P3 starts at 1.0 seconds
P4 starts at 1.5 seconds
...

Therefore, a driver's starting `totalTime` depends on the qualifying position.

Lap Times

During each normal racing lap, an active driver receives a random lap time within the range specified for the driver's team. The lap time should be added to the driver's accumulated race time.

Pit Stops

Every driver is assigned **one random pit-stop lap** between Lap 1 and Lap 55. **The two drivers from the same team cannot pit on the same lap**. When a driver reaches the assigned pit lap, generate a random pit-stop time using the team's pit-time range and add it to the driver's total race time. A driver who retires before reaching the scheduled pit lap does not need to complete the pit stop.

Retirement

Each driver receives a random retirement rate within the range specified for the team. During each normal racing lap, determine whether the driver retires.

Once a driver retires:

- the driver should not complete any additional laps
- the driver should no longer appear in the active race standings

---

Overtaking and Placement

The track is considered wide enough to allow unlimited overtaking. Therefore, you **do not need to create special overtaking code**. Drivers are ranked based on their accumulated `totalTime`. The active driver with the smallest `totalTime` is currently in first place.

This is a simplified racing model. You do not need to model real-world lapping, or other racing regulations.

Main Class

`Main.java` is **completely provided** and **should not be modified in Part I**.

It creates all 11 teams and starts the race.

Note: These values are for this simplified programming simulation and are not intended to represent real racing teams.

Example Output

Qualifying

===== QUALIFYING =====

1. Driver_name1 team1 87.702 seconds
2. Driver_name2 team2 87.853 seconds
3. Driver_name3 team1 87.906 seconds

...

Each lap in race

Lap 21
Driver_name1 pits: +21.742 seconds
Driver_name2 retired on Lap 21

1. Driver_name1 team1 gap: 0.000
2. Driver_name2 team2 gap: 3.126
3. Driver_name3 team1 gap: 3.877

…

Part I Submission — 60%

Part I must be completed **during the lab period**. At the end of the lab, submit your current versions of:

`Driver.java`
`Team.java`
`Race.java`

Your code does not need to be perfect or completely finished to receive credit. **Partial credit will be awarded for correctly completed portions.** Therefore, do not delete working code simply because the complete simulation is not finished.

For Part I, You may use:

- course lecture notes
- previous CS212 lab materials
- your own previous code
- Java documentation

You must **not use generative AI or code-generation tools during the in-lab portion**.

Part II – Safety Car Extension — 40%

After the lab, **extend your Part I program** by implementing Safety Car behavior.

You will create one additional class:

`SafetyCar.java`

Submit a `.zip` file containing:

```
Driver.java
Team.java
Race.java
SafetyCar.java
Main.java
```

The final submission contributes the remaining **40%** of the project grade.

Safety Car Rules

When a driver retires during normal racing: **30%** of retirements cause a Safety Car.

If a Safety Car is already active, another Safety Car should not be created.

Each Safety Car lasts a random number from **2–6 laps**.

Behavior During Safety Car

During a Safety Car lap:

- active drivers still accumulate Safety Car lap time
- drivers may still complete their scheduled pit stop
- drivers do not retire during Safety Car laps

After **each Safety Car lap**:

1. sort all active drivers by accumulated race time
2. compress the field so that drivers are **0.5** seconds apart

A driver who pits during the Safety Car may lose positions. However, because the field is compressed, the driver's final gap after rejoining the Safety Car queue will again follow the 0.5-second spacing.

This is a simplified racing model. You do not need to model real-world unlapping, or other racing regulations.

SafetyCar Class

Create a class that stores information about the current Safety Car.

For example:

```
private int remainingLaps
```

Your class should include methods that allow `Race` to determine:

- whether the Safety Car is still active
- how many laps remain
- when one Safety Car lap has been completed

Possible Changes to Race Class

You will likely need methods such as:

```
private void runSafetyCarLap(int lap)
private void compressField()
```